LAPORAN PRAKTIKUM 3

STACK

STRUKTUR DATA

Disusun Oleh:

Nama: Amiratul Fadhilah

NIM: 2511532023

Dosen Pengampu: Dr. Ir. Wahyudi, S. T., M. T

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

KATA PENGANTAR

Puji syukur ke hadirat Allah Yang Maha Esa atas rahmat dan karunia-Nya, sehingga penulisan laporan praktikum dengan judul “Stack” ini dapat diselesaikan tepat waktu. Laporan ini disusun sebagai salah satu syarat untuk memenuhi tugas mata kuliah Praktikum Struktur Data.

Dalam laporan ini, penulis akan memaparkan implementasi struktur data *stack* yang beroperasi berdasarkan prinsip *Last In First Out* (LIFO) menggunakan bahasa pemrograman Java. Fokus pembahasan mencakup perancangan *stack* secara statis menggunakan *array*, pemanfaatan koleksi dinamis *ArrayList*, hingga aplikasi nyatanya dalam mengevaluasi ekspresi matematika *postfix*.

Penulis menyadari bahwa masih terdapat berbagai kekurangan, baik dalam penyusunan narasi laporan maupun optimasi kode program di dalamnya. Oleh karena itu, kritik dan saran yang membangun dari dosen pengampu maupun asisten praktikum sangat diharapkan demi perbaikan di masa mendatang. Semoga dokumentasi praktikum ini dapat memberikan manfaat tambahan bagi pembaca.

Padang, 17 April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....i

DAFTAR ISI.....ii

BAB I PENDAHULUAN

1.1 Latar Belakang.....1
1.2 Tujuan Praktikum.....1
1.3 Manfaat Praktikum.....2

BAB II PEMBAHASAN

2.1 Landasan Teori 3
2.2 Langkah Praktikum.....4
2.3 Kode Program.....5
2.3.1 Kelas stackArray 5
2.3.2 Kelas Driver stackArray 6
2.3.3 Kelas StackPostfix.....7
2.3.4 Kelas NilaiMaksimum.....9
2.3.5 Kelas SiswaStack 10

BAB III PENUTUP

3.1 Kesimpulan.....13
3.2 Saran.....13

DAFTAR PUSTAKA.....14

TUGAS 3 PRAKTIKUM STRUKTUR DATA.....15

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, pengelolaan sekumpulan data memerlukan struktur yang efisien dan terorganisasi. *Stack* atau tumpukan muncul sebagai salah satu solusi yang mengadopsi prinsip *Last In First Out* (LIFO), di mana data yang paling akhir dimasukkan akan menjadi yang pertama kali diproses. Struktur ini sangat penting karena merepresentasikan banyak logika dunia nyata ke dalam sistem komputasi, memastikan bahwa alur data tetap konsisten dan terukur dalam satu jalur akses tunggal yaitu bagian *top*.

Di dalam arsitektur komputer modern, mekanisme operasi *Stack* memegang peranan yang sangat vital di balik layar. Struktur ini digunakan secara luas oleh sistem operasi untuk manajemen memori sementara, terutama saat melakukan pemanggilan fungsi bertingkat (*function call*) atau eksekusi fungsi rekursif. Selain itu, fitur-fitur aplikasi yang sering kita gunakan seperti tombol *Undo* (kembali ke aksi sebelumnya) pada teks editor, fitur *Back* pada *browser* internet, hingga proses *compiler* dalam mengevaluasi rumus matematika, semuanya bergantung pada keandalan struktur data ini.

Mengingat pentingnya peran tersebut, pemahaman teoritis dan teknis mengenai cara kerja *Stack* menjadi kompetensi wajib. Melalui praktikum ini, implementasi *Stack* akan dieksplorasi lebih jauh, baik menggunakan memori statis seperti *Array* maupun alokasi dinamis menggunakan *ArrayList*, sehingga penyelesaian berbagai permasalahan algoritmik dapat dilakukan dengan presisi dan efisien.

1.2 Tujuan Praktikum

Tujuan dari pelaksanaan praktikum ini adalah sebagai berikut:

1) Memahami konsep dasar struktur data *Stack* dan prinsip kerja LIFO secara teoritis maupun praktis.

2) Mampu mengimplementasikan *Stack* dari awal menggunakan tipe data statis berukuran tetap seperti *Array*.
3) Mampu mengimplementasikan *Stack* secara dinamis menggunakan kelas koleksi seperti *ArrayList*.
4) Menguasai pemakaian kelas bawaan java.util.Stack untuk mempercepat pengembangan program.
5) Menerapkan logika *Stack* ke dalam studi kasus nyata, seperti evaluasi ekspresi *Postfix* dan pencarian nilai maksimum.

1.3 Manfaat Praktikum

Melalui praktikum ini, manfaat yang diperoleh antara lain:

1) Melatih pola pikir sistematis dalam menyimpan dan mengakses data dengan aturan tertentu.
2) Memberikan gambaran bagaimana memori komputer bekerja di balik layar, khususnya saat menumpuk antrean proses.
3) Membekali kemampuan untuk memilih struktur data yang paling tepat ketika dihadapkan pada masalah yang membutuhkan pembacaan data secara terbalik (*reverse reading*).

BAB II

PEMBAHASAN

2.1 Landasan Teori

Stack (Tumpukan) adalah sebuah struktur data linear yang bersifat sekuensial dan beroperasi sepenuhnya dengan prinsip LIFO (Last In, First Out). Sifat LIFO ini menegaskan bahwa elemen terakhir yang dimasukkan ke dalam tempat penyimpanan akan menjadi elemen pertama yang dikeluarkan. Seluruh proses penyisipan elemen baru (penambahan data) dan penghapusan elemen (pengambilan data) dikunci dan hanya dapat dilakukan pada satu titik akses yang sama. Titik akses tunggal ini secara universal dikenal dengan istilah *Top* dari tumpukan.

Dalam implementasi pemrogramannya, *Stack* dapat dibangun menggunakan dua pendekatan memori yang berbeda, yaitu pendekatan statis dan dinamis. Pendekatan statis umumnya menggunakan *Array*, di mana ukuran maksimal tumpukan sudah harus ditentukan sejak awal program dijalankan dan tidak bisa diubah, sehingga lebih hemat proses namun kaku. Sebaliknya, pendekatan dinamis memanfaatkan struktur *Linked List* atau *Collection* (seperti ArrayList di Java), di mana ukuran tumpukan dapat membesar atau mengecil secara otomatis seiring dengan bertambah atau berkurangnya data yang dimasukkan, memberikan fleksibilitas memori yang jauh lebih tinggi.

Ketika bekerja dengan operasi *Stack*, terutama yang dibangun secara manual, terdapat dua macam kondisi kritis yang harus selalu diawasi oleh pemrogram. Pertama adalah Stack Overflow, yaitu sebuah kondisi *error* yang terjadi ketika kita memaksa memasukkan data baru ke dalam tumpukan yang kapasitas maksimalnya sudah penuh. Kedua adalah Stack Underflow, yakni kondisi kesalahan logika di mana program mencoba mengambil atau menghapus data dari tumpukan yang pada saat itu kondisinya sudah benar-benar kosong. Oleh karena itu, pengecekan status kosong atau penuh merupakan prosedur wajib sebelum operasi utama dilakukan.

Secara teknis, operasi-operasi utama pada *Stack* meliputi:

- Push: Fungsi ini bertugas memasukkan elemen baru ke bagian paling atas *Stack*. Posisi penunjuk *Top* akan naik.

- Pop: Fungsi ini bertugas mengambil sekaligus menghapus elemen dari bagian paling atas *Stack*. Posisi penunjuk *Top* akan turun.
- Peek (Top): Fungsi ini digunakan untuk melihat elemen teratas saat ini tanpa menghapusnya dari struktur tumpukan.
- isEmpty: Fungsi validasi (*boolean*) untuk memeriksa apakah *Stack* sedang dalam kondisi kosong.

2.2 Langkah Praktikum

1) Menyiapkan aplikasi IDE Java seperti Eclipse.
2) Membuat package baru bernama pekan3_2511532023.
3) Membuat lima *class* Java sesuai struktur program yang dibutuhkan untuk materi *Stack*, dengan rincian sebagai berikut:
    - stackArray_2511532023: *class* utama untuk memodelkan struktur data *Stack* secara manual menggunakan tipe data primitif *Array* (statis).
    - stackArrayDriver_2511532023: *class* penguji (*driver*) yang berisi *main method* untuk menjalankan dan memverifikasi fungsi *push*, *pop*, dan *peek* dari *Stack* statis sebelumnya.
    - StackPostfix_2511532023: *class* untuk mempraktikkan implementasi *Stack* dalam skenario nyata, yaitu mengevaluasi dan menghitung ekspresi matematika (logika *Postfix*).
    - NilaiMaksimum_2511532023: *class* khusus untuk menguji logika manipulasi data, yaitu mencari nilai tertinggi di dalam tumpukan menggunakan keranjang *Stack* cadangan.
    - SiswaStack_2511532023: *class* untuk membangun *Stack* yang lebih fleksibel dan dinamis menggunakan ArrayList, serta mempraktikkan penumpukan objek utuh (data Siswa).
4) Melakukan pengecekan dan eksekusi pada setiap *class* yang memiliki *main method*.
5) Menganalisis *output* program pada *console* dan memastikan seluruh operasi penumpukan berjalan sesuai dengan kaidah *Last In, First Out* (LIFO).

2.3 Kode Program

2.3.1 Kelas stackArray

```
package pekan3_2511532023;

public class stackArray_2511532023 {
	static final int MAX_2023 = 1000;
	int top_2023;
	int a_2023[] = new int [MAX_2023];
	boolean isEmpty_2023()
	{
		return (top_2023 < 0);
	}
	stackArray_2511532023()
	{
		top_2023 = -1;
	}
	boolean push_2023(int x)
	{
		if (top_2023 >= (MAX_2023 - 1)) {
			System.out.println("Stack Overflow");
			return false;
		}
		else {
			a_2023[++top_2023] = x;
			System.out.println(x + " dimasukkan dalam stack");
			return true;
		}
	}
	int pop_2023()
	{
		if (top_2023 < 0) {
			System.out.println("Stack Underflow");
			return 0;
		}
		else {
			int x = a_2023[top_2023--];
			return x;
		}
	}
	int peek_2023()
	{
		if (top_2023 < 0) {
			System.out.println("Stack Underflow");
			return 0;
		}
		else {
			int x = a_2023[top_2023];
			return x;
		}
	}
	void print_2023() {
		for(int i = top_2023; i > -1; i--) {
			System.out.print(" " + a_2023[i]);
		}
	}
```

```
}
```

Program ini adalah bentuk dasar pembuatan struktur data *Stack*. Pembuatan ruang penyimpanan dilakukan menggunakan tipe data primitif *Array* dengan batas maksimal 1000 elemen. Variabel top_2023 bertugas memegang posisi indeks teratas. Dilengkapi juga dengan gerbang logika pengamanan Overflow dan Underflow agar sistem tidak mogok (*crash*) jika terjadi kesalahan input batasan.

2.3.2 Kelas Driver stackArray

```
package pekan3_2511532023;

public class stackArrayDriver_2511532023 {
	public static void main(String[] args) {
		stackArray_2511532023 s = new stackArray_2511532023();
		s.push_2023(10);
		s.push_2023(20);
		s.push_2023(30);
		System.out.println(s.pop_2023() + " dikeluarkan dari 
stack");
		System.out.println("Elemen teratas adalah: " + 
s.peek_2023());
		System.out.print("Elemen pada stack :");
		s.print_2023();
	}
}
```

Kelas ini berfungsi sebagai *Driver* atau program utama pengeksekusi untuk menguji logika dari kelas stackArray yang telah dibuat sebelumnya. Melalui kelas ini, objek *Stack* diinisialisasi dan diisi dengan angka 10, 20, dan 30 secara berurutan. Pemanggilan fungsi pop() dan peek() pada kelas ini secara langsung membuktikan bahwa prinsip LIFO benar-benar berjalan, di mana angka terakhir yang dimasukkan akan ditarik paling awal.

Output:

```
Console
<terminated> stackArrayDriver_251153202
10 dimasukkan dalam stack
20 dimasukkan dalam stack
30 dimasukkan dalam stack
30 dikeluarkan dari stack
Elemen teratas adalah: 20
Elemen pada stack : 20 10
```

2.3.3 Kelas StackPostfix

```
package pekan3_2511532023;



import java.util.Scanner;

import java.util.Stack;



public class StackPostfix_2511532023 {

	public static int postfixEvaluate_2023(String expression) {

		Stack<Integer> s_2023 = new Stack<Integer>();

		Scanner input_2023 = new Scanner(expression);

		while (input_2023.hasNext()) {

			if (input_2023.hasNextInt()) {

				s_2023.push(input_2023.nextInt());

			} else {

				String operator_2023 = input_2023.next();

				int operand2_2023 = s_2023.pop();

				int operand1_2023 = s_2023.pop();

				if (operator_2023.equals("+")) {

					s_2023.push(operand1_2023 +
operand2_2023);
```

```
                        }
                        else if (operator_2023.equals("-")) {
                              s_2023.push(operand1_2023 -
operand2_2023);
                        } else if (operator_2023.equals("*")) {
                              s_2023.push(operand1_2023 *
operand2_2023);
                        } else {
                              s_2023.push(operand1_2023 /
operand2_2023);
                        }
                  }
            }
            input_2023.close();
            return s_2023.pop();
      }
      public static void main(String[] args) {
            System.out.println("hasil postfix = " +
postfixEvaluate_2023("5 2 4 * +  7 -"));
      }
}
```

Kode ini adalah pemanfaatan *Stack* untuk menyelesaikan masalah komputerisasi secara nyata. Dengan menggunakan struktur data bawaan java.util.Stack, program membaca kalimat matematika dari kiri ke kanan. Setiap kali ia menemukan angka, angka itu akan ditumpuk (push). Namun saat mendeteksi simbol operator (seperti + atau *), ia menarik paksa dua angka terakhir dari tumpukan (pop), mengeksekusi perhitungannya, dan menempatkan hasilnya kembali ke atas tumpukan.

Output:

```
Console
<terminated> StackPostfix_2
hasil postfix = 6
```

2.3.4 Kelas NilaiMaksimum

```java
package pekan3_2511532023;

import java.util.Stack;

public class NilaiMaksimum_2511532023 {
	public static int max_2023(Stack<Integer> s_2023) {
		Stack<Integer> backup_2023 = new Stack<Integer>();
		int maxValue_2023 = s_2023.pop();
		backup_2023.push(maxValue_2023);
		while (!s_2023.isEmpty()) {
			int next_2023 = s_2023.pop();
			backup_2023.push(next_2023);
			maxValue_2023 = Math.max(maxValue_2023, next_2023);
		}
		while (!backup_2023.isEmpty()) {
			s_2023.push(backup_2023.pop());
		}
		return maxValue_2023;
	}
	public static void main(String[] args) {
		Stack<Integer> s_2023 = new Stack<Integer>();
		s_2023.push(70);
		s_2023.push(12);
		s_2023.push(20);
		System.out.println("Isi stack " + s_2023);
		System.out.println("Stack Teratas " + s_2023.peek());
		System.out.println("Nilai maksimum " + max_2023(s_2023));
	}
}
```

Program ini merupakan penerapan logika manipulasi yang sangat menarik. Sesuai aturan fundamental *Stack* yang melarang pemrogram mengintip data secara sembarangan di bagian tengah tumpukan, kita diwajibkan menggunakan ruang "cadangan" (backup_2023) untuk menemukan data bernilai tertinggi. Seluruh isi tumpukan dikeluarkan satu per satu (*pop*) dan dipindahkan ke cadangan, sembari nilai terbesarnya dicatat. Setelah nilai maksimum ditemukan, semua data dikembalikan ke tumpukan asalnya agar tidak hilang.

Output:

```
Console
<terminated> NilaiMaksimum_2511532
Isi stack [70, 12, 20]
Stack Teratas 20
Nilai maksimum 70
```

2.3.5 Kelas SiswaStack

```java
package pekan3_2511532023;
import java.util.ArrayList;
class Siswa_2023 {
	String nama_2023;
	int nim_2023;

	public Siswa_2023(String nama_2023, int nim_2023) {
		this.nama_2023 = nama_2023;
		this.nim_2023 = nim_2023;
	}

	@Override
	public String toString() {
		return "Nim: " + nim_2023 + ", Nama: " + nama_2023;
	}
}
public class SiswaStack_2511532023 {
	private ArrayList<Siswa_2023> stack_2023;

	public SiswaStack_2511532023() {
		stack_2023 = new ArrayList<>();
	}

	public void push(Siswa_2023 mhs) {
		stack_2023.add(mhs);
	}

	public Siswa_2023 pop_2023() {
		if (!isEmpty()) {
			return stack_2023.remove(stack_2023.size() - 1);
		}
		return null;
	}

	public Siswa_2023 peek_2023() {
		if (!isEmpty()) {
			return stack_2023.get(stack_2023.size() - 1);
		}
		return null;
	}

	public boolean isEmpty() {
		return stack_2023.isEmpty();
```

```
        }

        public void tampilkanSiswa() {
                for (int i = stack_2023.size() - 1; i >= 0; i--) {
                        System.out.println(stack_2023.get(i));
                }
        }
        public static void main(String[] args) {
                SiswaStack_2511532023 studentStack_2023 = new
SiswaStack_2511532023();

                Siswa_2023 mhs1 = new Siswa_2023("Ali", 1);
                Siswa_2023 mhs2 = new Siswa_2023("Boby", 2);
                Siswa_2023 mhs3 = new Siswa_2023("Charles", 3);

                studentStack_2023.push(mhs1);
                studentStack_2023.push(mhs2);
                studentStack_2023.push(mhs3);

                System.out.println("Siswa di dalam stack: ");
                studentStack_2023.tampilkanSiswa();

                System.out.println("Siswa teratas " +
studentStack_2023.peek_2023());
                System.out.println("Mengeluarkan siswa teratas dari stack:
" + studentStack_2023.pop_2023());
                System.out.println("Daftar siswa setelah di pop: ");
                studentStack_2023.tampilkanSiswa();
        }
}
```

Berbeda dengan pembatasan tumpukan pada ukuran baku (statis), kelas ini memanfaatkan ArrayList yang sifatnya dinamis, sehingga ukurannya bisa membesar atau menyusut secara otomatis menyesuaikan isi data. Lebih jauh, kode ini mendemonstrasikan bahwa *Stack* tidak hanya terbatas untuk menampung tipe dasar berupa angka (integer), melainkan juga bisa menumpuk dan memanajemen keseluruhan entitas objek utuh (dalam hal ini adalah data Siswa).

Output:

```
Console
<terminated> SiswaStack_2511532023 [Java Application] C:\Users\ACER\.p2\pool\p
Siswa di dalam stack:
Nim: 3, Nama: Charles
Nim: 2, Nama: Boby
Nim: 1, Nama: Ali
Siswa teratas Nim: 3, Nama: Charles
Mengeluarkan siswa teratas dari stack: Nim: 3, Nama: Charles
Daftar siswa setelah di pop:
Nim: 2, Nama: Boby
Nim: 1, Nama: Ali
```

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan serangkaian uji coba dan penulisan kode pada praktikum ini, dapat disimpulkan bahwa struktur data *Stack* terbukti sangat penting dan dapat diandalkan dalam skenario komputasi yang mengharuskan pemrosesan riwayat operasi secara terbalik. Karakteristik utamanya yang berpegang teguh pada prinsip LIFO (*Last In, First Out*) memastikan bahwa elemen informasi yang paling terakhir masuk akan dilayani terlebih dahulu. Aturan tunggal pada *Top* ini menyederhanakan pelacakan urutan data secara signifikan tanpa perlu melakukan iterasi yang rumit pada keseluruhan memori.

Kesimpulan akhir yang dapat ditarik adalah pemahaman teori *Stack* tidak hanya berhenti pada konsep penumpukan benda mati, melainkan merupakan landasan bagi penyelesaian berbagai permasalahan algoritmik nyata. Ini terbukti melalui keberhasilan simulasi evaluasi operasi matematika kompleks (*Postfix*) tanpa memerlukan tanda kurung, serta kemampuan melakukan penelusuran pencarian nilai tertinggi (*Max Value*) dengan memanfaatkan logika keranjang cadangan guna menjaga keutuhan susunan tumpukan data awal.

3.2 Saran

Dalam pengembangan program tingkat lanjut yang lebih kompleks, disarankan untuk mulai mengeksplorasi penggunaan *Interface Deque* (seperti ArrayDeque) dibandingkan java.util.Stack konvensional. Hal ini dikarenakan secara performa dan pembagian *thread* pada komputasi modern, *Deque* jauh lebih efisien. Selain itu, implementasi *error handling* menggunakan blok instruksi *Custom Exceptions* (*Try-Catch*) sangat disarankan untuk menggantikan format peringatan teks terminal biasa saat sistem mendeteksi kendala seperti memori penuh atau tumpukan kosong.

DAFTAR PUSTAKA

[1] W. Imelda, "Laporan Praktikum Struktur Data: Stack (Tumpukan)," 2024. [Daring]. Tersedia pada: https://www.scribd.com/document/890879817/211810101095-WindaImelda-SD-KLS-B-LP9. [Diakses: 17-Apr-2026].

[2] Universitas Trunojoyo Madura, "Laporan Praktikum Stack (Tumpukan)," 2022. [Daring]. Tersedia pada: https://www.studocu.id/id/document/universitas-trunojoyo-madura/struktur-data/laporan-praktikum-stack-tumpukan/45027872. [Diakses: 17-Apr-2026].

[3] I. Ikhwan, "Praktikum SDA - Stack/Tumpukan," 2020. [Daring]. Tersedia pada: https://www.scribd.com/document/482433950/IchsnaulIkhwan-2TIA-PrakSDA-Stack-Tumpukan. [Diakses: 17-Apr-2026].

[4] Tim Pengajar Struktur Data, "Jobsheet 7 - Stack," 2024. [Daring]. Tersedia pada: https://www.scribd.com/document/731764462/Jobsheet-7-Stack. [Diakses: 18-Apr-2026].

TUGAS 3 PRAKTIKUM STRUKTUR DATA

1. **Soal**

   Membuat program yang mensimulasikan perpindahan halaman web. Halaman yang terakhir dibuka akan berada di posisi teratas tumpukan, dan menjadi halaman pertama yang dihapus saat tombol "Back" ditekan.

   a. Kelas ADT Website (Website_NIMLengkap)

      Atribut:

      - Judul Website
      - URL Website

      Method:

      - Constructor untuk menginisialisasi atribut.
      - Selektor (Getter) untuk setiap atribut.
      - Mutator (Setter) untuk setiap atribut.

   b. Kelas Driver (Browser_NIMLengkap)

      - Kunjungi Website (Push) : Memasukkan data website baru yang sedang dikunjungi ke dalam tumpukan
      - Tombol Back (Pop) : Menghapus halaman website terakhir (paling atas) dan menampilkan judul halaman yang berhasil dihapus.
      - Lihat Halaman Saat Ini (Peek) : Menampilkan detail website yang sedang aktif dibuka tanpa menghapusnya dari tumpukan.
      - Cek Status History : Menampilkan jumlah total riwayat yang tersimpan atau mengecek apakah riwayat kosong menggunakan .isEmpty().

2. **Jawab**

   - Kelas ADT Website (Website_2511532023)

     Kelas Website berfungsi murni sebagai cetakan data untuk mendefinisikan sebuah halaman web. Tugas utamanya hanyalah menyimpan informasi spesifik dari situs yang sedang dibuka, yaitu sekadar judul dan tautan URL-nya, agar data tersebut dibungkus dengan rapi dan siap dipanggil kapan pun program membutuhkannya.

```java
package pekan3_2511532023;

public class Website_2511532023 {
	private String judul_2023;
	private String url_2023;

	public Website_2511532023(String judul_2023, String url_2023) {
		this.judul_2023 = judul_2023;
		this.url_2023 = url_2023;
	}

	public String getJudul_2023() {
		return judul_2023;
	}
	public String getUrl_2023() {
		return url_2023;
	}

	public void setJudul_2023(String judul_2023) {
		this.judul_2023 = judul_2023;
	}
	public void setUrl_2023(String url_2023) {
		this.url_2023 = url_2023;
	}
}
```

- Kelas Driver (Browser_2511532023)

  Kelas Browser bertindak sebagai mesin utama sekaligus pusat berjalannya menu pilihanmu. Di sinilah program memanfaatkan struktur data bernama Stack atau tumpukan untuk menyimpan riwayat kunjungan web tersebut, di mana setiap halaman baru akan selalu ditaruh di posisi paling atas, sehingga ketika tombol kembali atau *Back* ditekan, program akan otomatis menghapus halaman web yang paling terakhir dibuka sesuai dengan prinsip *Last In, First Out* (LIFO).

```java
package pekan3_2511532023;

import java.util.Stack;
import java.util.Scanner;

public class Browser_2511532023 {
	public static void main(String[] args) {
		Stack<Website_2511532023> history = new Stack<>();
		Scanner sc = new Scanner(System.in);
		int pilihan;

		do {
			System.out.println("\n=== Browser History NIM:
2511532023 ===");
			System.out.println("1. Kunjungi Website (Push)");
```

```
                    System.out.println("2. Tombol Back (Pop)");
                    System.out.println("3. Lihat Halaman Aktif (Peek)");
                    System.out.println("4. Keluar");
                    System.out.print("Pilihan: ");
                    pilihan = sc.nextInt();
                    sc.nextLine();

                    switch (pilihan) {
                    case 1:
                          System.out.print("Masukkan Judul: ");
                          String judul = sc.nextLine();

                          System.out.print("Masukkan URL: ");
                          String url = sc.nextLine();

                          history.push(new Website_2511532023(judul,
url));
                          System.out.println("Berhasil mengunjungi
halaman!");
                          break;

                    case 2:
                          if (history.isEmpty()) {
                                System.out.println("Gagal Kembali!
Riwayat browser saat ini masih kosong.");
                          } else {
                                Website_2511532023 halamanDihapus =
history.pop();
                                System.out.println("Berhasil Kembali!
Anda telah meninggalkan halaman: " + halamanDihapus.getJudul_2023());
                          }
                          break;

                    case 3:
                          if (history.isEmpty()) {
                                System.out.println("Belum ada website
yang sedang dibuka.");
                          } else {
                                Website_2511532023 halamanAktif =
history.peek();
                                System.out.println("\n--- Halaman Saat
Ini ---");
                                System.out.println("Judul : " +
halamanAktif.getJudul_2023());
                                System.out.println("URL   : " +
halamanAktif.getUrl_2023());
                                System.out.println("Total riwayat
tersimpan: " + history.size() + " halaman.");
                          }
                          break;

                    case 4:
                          System.out.println("Menutup browser.");
                          break;

                    default:
```

```
                            System.out.println("Pilihan tidak valid.
Silakan coba lagi.");
                        }
                } while (pilihan != 4);
                sc.close();
        }
}
```

Output:

- Mengunjungi Website (Push)

```
=== Browser History NIM: 2511532023 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilihan: 1
Masukkan Judul: SSO UNAND
Masukkan URL: https://sso.unand.ac.id/login
Berhasil mengunjungi halaman!
```

- Melihat Halaman Aktif (Peek)

```
=== Browser History NIM: 2511532023 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilihan: 3

--- Halaman Saat Ini ---
Judul : SSO UNAND
URL   : https://sso.unand.ac.id/login
Total riwayat tersimpan: 1 halaman.
```

- Menekan Tombol Back (Pop)

```
=== Browser History NIM: 2511532023 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilihan: 2
Berhasil Kembali! Anda telah meninggalkan halaman: SSO UNAND
```

- Keluar

```
=== Browser History NIM: 2511532023 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Keluar
Pilihan: 4
Menutup browser.
```